

Casos de prueba del intérprete

A continuación se presentan los cinco casos de prueba ejecutados para verificar el funcionamiento del lenguaje de comandos de la casa inteligente y del patrón de diseño Interpreter.

Resultado en Consola:

```
run:
===== CASO 1 =====
Accion: Encendiendo la luz.
Estado -> Luz: ENCENDIDA | Puerta: CERRADA

===== CASO 2 =====
Accion: Encendiendo la luz.
Accion: Abriendo la puerta.
Estado -> Luz: ENCENDIDA | Puerta: ABIERTA

===== CASO 3 =====
Condicion: La luz esta encendida.
Accion: Abriendo la puerta.
Estado -> Luz: ENCENDIDA | Puerta: ABIERTA

===== CASO 4 =====
Condicion: La luz esta apagada.
Accion: Encendiendo la luz.
Estado -> Luz: ENCENDIDA | Puerta: CERRADA

===== CASO 5 =====
Accion: Encendiendo la luz.
Condicion: La luz esta encendida.
Accion: Abriendo la puerta.
Condicion: La luz esta encendida.
Accion: Apagando la luz.
Estado -> Luz: APAGADA | Puerta: ABIERTA
BUILD SUCCESSFUL (total time: 0 seconds)
```

Caso de prueba 1. Expresión terminal: ENCENDER_LUZ

Expresión interpretada: ENCENDER_LUZ

Contexto inicial:

- Luz: apagada.
- Puerta: cerrada.

Resultado esperado: La luz cambia de apagada a encendida y la puerta permanece cerrada.

Caso de prueba 2. Expresión compuesta: ENCENDER_LUZ + ABRIR_PUERTA

Expresión interpretada: SECUENCIA(ENCENDER_LUZ, ABRIR_PUERTA)

Contexto inicial:

- Luz: apagada.
- Puerta: cerrada.

Resultado esperado: Se ejecutan las instrucciones en el orden definido: primero se enciende la luz y luego se abre la puerta.

Caso de prueba 3. Expresión condicional con luz encendida

Expresión interpretada: SI_LUZ_ENCENDIDA ENTONCES ABRIR_PUERTA SINO ENCENDER_LUZ

Contexto inicial:

- Luz: encendida.
- Puerta: cerrada.

Resultado esperado: Como la luz ya está encendida, la condición toma la rama verdadera y ejecuta la acción de abrir la puerta.

Caso de prueba 4. Expresión condicional con luz apagada

Expresión interpretada: SI_LUZ_ENCENDIDA ENTONCES ABRIR_PUERTA SINO ENCENDER_LUZ

Contexto inicial:

- Luz: apagada.
- Puerta: cerrada.

Salida en consola:

Resultado esperado: Como la luz está apagada, la condición toma la rama falsa y enciende la luz.

Caso de prueba 5. Expresión anidada con cambio de estado del Context

Expresión interpretada:

```
SECUENCIA(  
  ENCENDER_LUZ,  
  SI_LUZ_ENCENDIDA ENTONCES  
    SECUENCIA(  
      ABRIR_PUERTA,  
      SI_LUZ_ENCENDIDA ENTONCES APAGAR_LUZ  
      SINO ABRIR_PUERTA  
    )  
  SINO ENCENDER_LUZ  
)
```

Contexto inicial:

- Luz: apagada.
- Puerta: cerrada.

Resultado esperado: La primera instrucción modifica el contexto al encender la luz. La condición posterior utiliza ese nuevo estado para ejecutar la rama verdadera, y la condición anidada vuelve a evaluar el contexto actualizado antes de apagar la luz. Este caso demuestra que el estado del Context cambia durante la interpretación y afecta las decisiones posteriores.

Explicación de las clases del patrón Interpreter

En este proyecto, las clases terminales son `EncenderLuzExpression`, `ApagarLuzExpression` y `AbrirPuertaExpression`, ya que representan acciones individuales del lenguaje y ejecutan directamente una operación sobre el contexto. Las clases no terminales son `SecuenciaExpression`, que interpreta varias expresiones en un orden determinado, y `SiLuzEncendidaExpression`, que interpreta una de dos expresiones dependiendo del estado del contexto. El Context almacena la información compartida de la casa inteligente mediante los atributos `luzEncendida` y `puertaAbierta`. Este objeto viaja por todas las expresiones durante la interpretación, permitiendo que unas instrucciones modifiquen el estado y que otras tomen decisiones utilizando ese estado actualizado, lo que representa el funcionamiento principal del patrón Interpreter.